

École Nationale Supérieure Polytechnique de Yaoundé

Université de Yaoundé I

PROJET ÉVALUÉ — APPLICATION DÉPLOYÉE SUR AWS

Déployer, tester, supprimer

Guide du dépôt (README)

Éditeur de documents partagé pour le journal d'une association étudiante

26 octobre 2026

Sommaire

Architecture en bref

Prérequis

Déployer

Tester

Supprimer

Structure du dépôt

Application web d'écriture à plusieurs pour le journal d'une association étudiante, déployée sur AWS (eu-north-1) : édition en temps réel avec mise en forme, présence et curseurs, historique des versions, liens de téléchargement PDF limités en nombre, crédit de 10 000 caractères par jour, anti-abus, tableau de bord et e-mail au créateur d'un lien à son dernier téléchargement.

Livrable	Où
Code	`packages/` (`shared` , `api` , `realtime` , `web`)
Infrastructure	`infra/` (AWS CDK v2, TypeScript)
Tests	`tests/` , `packages/*/test` , `infra/test` ; résultats bruts dans `tests/results/`
Dossier de conception	`docs/dossier-conception.md`
Rapport de test (preuves des deux exigences éliminatoires)	`docs/rapport-test.md`
Description de l'API	`docs/openapi.yaml` (HTTP, OpenAPI 3) et `docs/api-temps-reel.md` (WebSocket `/collab`)
Diagrammes	`docs/diagrammes/` (sources Mermaid `.mmd` et images `.png`)
Versions PDF des documents	`docs/pdf/`
Preuve de coût	`docs/cout.md`

Architecture en bref

```
Navigateur ──HTTPS──► CloudFront ──► ALB (n'accepte que CloudFront)
                                     ├── /collab (WebSocket) ──► serveur temps réel (1 × t3.small)
                                     │                           Hocuspocus + Yjs, journal,
compaction, worker PDF
                                     └── le reste ─────────────► API sans état (ASG t3.micro, 1 à 4,
2 zones)
                                                                 + front statique
DynamoDB (8 tables, à la demande) · S3 (instantanés, PDF, versionné) · SQS (demandes de PDF)
SSM (secrets) · SES (e-mail) · CloudWatch Logs
```

Une modification est **acceptée** quand elle est écrite dans DynamoDB (journal + crédit, dans une seule transaction) ; rien n'est diffusé ni confirmé avant. Un clic sur un lien **réserve** un exemplaire par un décrément atomique conditionnel : jamais un de plus. Détails et justifications : dossier de conception.

Prérequis

- Un compte AWS et un **utilisateur IAM** (jamais le compte root) avec AWS CLI v2 configurée (`aws configure` , région `eu-north-1`).
- Node.js 20 et npm ; `openssl` (génération des secrets).
- Pour les tests locaux : Docker (DynamoDB Local) ou Java 17 (jar DynamoDB Local).
- Pour le test de charge : rien en local (k6 tourne sur une instance EC2 dédiée).

Déployer

```
scripts/deploy.sh
# avec l'e-mail au créateur d'un lien (SES en bac à sable : expéditeur ET destinataires vérifiés) :
SES_SENDER=moi@exemple.org scripts/deploy.sh
```

Le script crée les secrets dans SSM (`/editeur/hmac-secret` , `/editeur/admin-token` , SecureString, jamais affichés), trouve la liste de préfixes CloudFront, compile les bundles, amorce CDK si besoin et déploie la stack `EditeurPartage` . Il affiche l'URL (`https://xxxx.cloudfront.net`) ; comptez 2 à 4 minutes de démarrage des instances.

- Application : `https://xxxx.cloudfront.net` (un nom et un e-mail suffisent pour entrer).
- Tableau de bord : `https://xxxx.cloudfront.net/admin` , jeton : `aws ssm get-parameter --region eu-north-1 --name /editeur/admin-token --with-decryption --query Parameter.Value --output text`

Mise à jour du code : relancer `scripts/deploy.sh` (le serveur temps réel est remplacé par une mise à jour progressive qui arrête l'ancien avant de lancer le nouveau : quelques secondes de reconnexion, rien d'accepté n'est perdu).

Tester

En local (sans AWS)

```
npm ci
npm run local:db            # DynamoDB Local (Docker) sur le port 8000
npm test                    # compilation + tous les tests (unitaires, intégration, scénarios, CDK)
npm run dev                 # application complète sur http://localhost:3000 (admin : jeton « admin-
        local »)
```

Scripts de mesure (résultats bruts horodatés dans `tests/results/`) :

Exigence	Local	Sur AWS
Nombre exact de téléchargements	`npm run test:c1`	`npm run test:c1 -- --base-url <URL>` [1]
Aucune modification perdue, convergence	`npm run test:c2`	`npm run test:c2 -- --base-url <URL> --bucket <BucketName> [--flush-delay 200]`
PDF à jour au moment du clic	`npm run test:c3`	`npm run test:c3 -- --base-url <URL>`
Crédit quotidien, plusieurs appareils	(Vitest, `tests/c2/c2.test.ts`)	`npm run test:c4 -- --base-url <URL> --bucket <BucketName>`

[1] À cause de l'anti-abus, 1 000 clics depuis un même poste sont bloqués (429) au-delà de 30/N requêtes en 10 s : lancer ce test depuis l'instance de test de charge (adresse exemptée).

`<BucketName>` : sortie `BucketName` de la stack (`aws cloudformation describe-stacks --region eu-north-1 --stack-name EditeurPartage --query "Stacks[0].Outputs"`).

Variante « arrêt brutal » du test d'écriture : pendant `npm run test:c2 -- --base-url … --bucket … --duration 120`, terminer l'instance temps réel :

```
ASG=$(aws cloudformation describe-stacks --region eu-north-1 --stack-name EditeurPartage --query
        "Stacks[0].Outputs[?OutputKey=='RealtimeAsgName'].OutputValue" --output text)
ID=$(aws autoscaling describe-auto-scaling-groups --region eu-north-1 --auto-scaling-group-names
        "$ASG" --query "AutoScalingGroups[0].Instances[0].InstanceId" --output text)
aws ec2 terminate-instances --region eu-north-1 --instance-ids "$ID"
```

L'ASG relance une instance ; les clients se reconnectent et renvoient ce qui n'avait pas été confirmé ; le script vérifie que chaque séquence confirmée est dans l'état final.

Test de charge (pic d'affluence, annonce de l'épuisement)

```
npm run build
(cd infra && npx cdk deploy EditeurPartageCharge)       # instance k6 c7i-flex.large, aucun port
        ouvert
```

Puis, dans une session Session Manager sur l'instance (console EC2 → Se connecter → Gestionnaire de sessions) :

```
sudo -iu ec2-user
nohup /opt/charge/run.sh https://xxxx.cloudfront.net 5000 12 > ~/charge.log 2>&1 &
tail -f ~/charge.log        # ~22 min : lien de 5 000 exemplaires ouvert dans 12 min,
                            # 300 rédacteurs, puis 15 000 visiteurs (80 % la première minute)
```

Rapatrier les résultats et supprimer la stack de test :

```
B=$(aws cloudformation describe-stacks --region eu-north-1 --stack-name EditeurPartageCharge --query
        "Stacks[0].Outputs[?OutputKey=='ResultsBucket'].OutputValue" --output text)
aws s3 cp "s3://$B/" tests/results/load/ --recursive
aws s3 rm "s3://$B" --recursive && (cd infra && npx cdk destroy EditeurPartageCharge)
```

Supprimer

```
scripts/destroy.sh          # demande de taper « supprimer »
```

Le script vide le bucket versionné (toutes les versions et marqueurs de suppression), détruit la stack de test de charge si elle existe puis la stack principale, supprime les secrets SSM et liste les ressources encore étiquetées `projet=editeur-partage` (la liste doit être vide ; l'index des étiquettes peut garder quelques minutes des ressources supprimées).

Reste volontairement en place : la stack `CDKToolkit` (amorçage CDK, partagée par tous les projets CDK du compte). Pour la supprimer aussi, vider son bucket `cdk-hnb659fds-assets-<compte>-eu-north-1` puis `aws cloudformation delete-stack --region eu-north-1 --stack-name CDKToolkit`.

Structure du dépôt

```
packages/shared    types, schéma DynamoDB, HMAC (sessions, tickets), protocole temps réel, formats
packages/api       API Fastify sans état : sessions, documents, liens, clic, admin, anti-abus
visiteurs
packages/realtime  Hocuspocus : group commit DynamoDB, crédit, refus, compaction, versions, PDF
(pdfkit)
packages/web       front sans framework : Quill 2 + y-quill, page de téléchargement, tableau de bord
infra              MainStack (EditeurPartage) et LoadTestStack (EditeurPartageCharge)
tests              c1, c2, c3, c4, load (k6 + rédacteurs), helpers, results (résultats bruts)
scripts            deploy.sh, destroy.sh, build, environnement local
```